



UNIVERSIDADE DE
VASSOURAS

ENGENHARIA DE SOFTWARE



UNIVERSIDADE DE VASSOURAS

Curso de Graduação em Engenharia de Software

Prof. Márcio Garrido

Mestre em Engenharia Elétrica e Telecomunicações

Especialista em Segurança da Informação

Especialista em Engenharia Elétrica e Automação

MBA Engenharia Segurança do Trabalho

Bacharel em Engenharia de Software

**Bacharel em Sistema da Informação e Analista de
Sistemas**



E-mail:

marcio.garrido@univassouras.edu.br



Disciplina: Laboratório de Programação Full Stack



Agenda da Aula

Disciplina: Laboratório de Programação Full Stack

Aula 4 — Django na Prática I: do zero ao primeiro model

1. Ambiente



venv e instalação do Django

2. Projeto e app



criar e entender a estrutura

3. Primeira view



o servidor no ar

4. Models



modelar dados em Python

5. Migrações



levar o model ao banco

6. Django Admin



um painel de graça

O Objetivo de Hoje

Ao fim da aula, você terá isto funcionando



Um projeto Django rodando com dados reais

Você vai criar um projeto do zero, definir seu primeiro model, levá-lo ao banco PostgreSQL e cadastrar registros por um painel administrativo — tudo sem escrever uma linha de HTML ou SQL ainda.



Mão na massa

Esta é a primeira aula prática. Acompanhe digitando junto — aprender Django é fazendo.



Usaremos o exemplo "Biblioteca"

Um cadastro de livros nos guiará; o mesmo raciocínio serve para qualquer projeto que você escolher.

Passo 1

Preparar o Ambiente

Antes do Django, um ambiente isolado só para o projeto

Cada projeto Python tem suas próprias dependências. O ambiente virtual (venv) evita que um projeto atrapalhe o outro.



Por que um ambiente virtual

Sem ele, todas as bibliotecas ficam misturadas no sistema. Com o venv, cada projeto tem sua "caixa" isolada — você pode ter um projeto em Django 5 e outro em Django 4 sem conflito.

Criando o Ambiente e Instalando o Django

Quatro comandos no terminal

WINDOWS (PowerShell / CMD)

```
# 1. criar o ambiente virtual  
python -m venv venv
```

```
# 2. ativar  
venv\Scripts\activate
```

macOS / LINUX (bash/zsh)

```
# 1. criar o ambiente virtual  
python3 -m venv venv
```

```
# 2. ativar  
source venv/bin/activate
```

COM O AMBIENTE ATIVO (mesmo comando nos dois sistemas)

```
# 3. instalar o Django  
pip install django
```

```
# 4. conferir a versão  
django-admin --version
```



O prompt muda

Depois de ativar, aparece (venv) no início da linha — sinal de que o ambiente está ligado. No Windows o comando de ativar usa \ (contrabarra); no macOS/Linux usa o comando source. Instale o Django sempre com o venv ativo.

Criando o Projeto

Um comando gera toda a estrutura

terminal

```
django-admin startproject biblioteca .  
python manage.py runserver
```



O que foi criado

Uma pasta biblioteca/ com as configurações e um arquivo manage.py — seu "controle remoto" do projeto.



O servidor no ar

runserver liga um servidor local. Abra <http://127.0.0.1:8000> e veja o foguete do Django decolar.

django-admin × manage.py

Duas portas de entrada para os mesmos comandos

Você acabou de usar os dois: django-admin para criar o projeto, manage.py para tudo o mais. Qual é a diferença entre eles?

django-admin

Comando genérico do Django, **não sabe qual projeto** está usando.

Só é realmente necessário para **criar** um projeto novo (startproject).

manage.py

O mesmo comando, mas **já configurado** para o SEU projeto (sabe o settings.py certo).

É o que você usa **no dia a dia** — quase tudo passa por ele.



Regra prática

Fora de um projeto (ou para criar um novo), use django-admin. Dentro de um projeto, sempre python manage.py — ele já sabe qual configuração usar.

Os Comandos do manage.py Mais Usados

Sua referência para o dia a dia no laboratório

Comando	O que faz
runserver	liga o servidor de desenvolvimento local
startapp <nome>	cria um novo app dentro do projeto
makemigrations	gera as migrações a partir dos models alterados
migrate	aplica as migrações pendentes no banco de dados
createsuperuser	cria um usuário administrador para o Django Admin
shell	abre um Python já conectado ao projeto (ORM pronto)
showmigrations	lista as migrações e quais já foram aplicadas
collectstatic	reúne os arquivos estáticos (CSS/JS/imagens) para produção
test	executa os testes automatizados do projeto

A Estrutura Gerada

O que cada arquivo faz

Arquivo	Para que serve
manage.py	Executa comandos do projeto (runserver, migrate...)
settings.py	Configurações: banco de dados, apps instalados, idioma
urls.py	Mapa de rotas: qual URL leva a qual view
wsgi.py / asgi.py	Ponte com o servidor quando o projeto for para produção



O settings.py é o coração

É lá que você registra apps, configura o PostgreSQL e ajusta idioma e fuso. Vamos visitá-lo várias vezes.

Projeto × App

Uma distinção importante do Django

PROJETO

É o site inteiro.

Guarda as configurações gerais e reúne todos os apps.

Ex.: biblioteca

APP

É um módulo do site.

Cuida de uma funcionalidade específica, com seus models e views.

Ex.: acervo, usuarios



A regra prática

Um projeto tem vários apps. Cada app resolve uma parte — e pode até ser reaproveitado em outro projeto.

Criando o Primeiro App

O app "acervo"

terminal

```
python manage.py startapp acervo
```

Depois, registre o app no settings.py para o Django reconhecê-lo:

settings.py

```
INSTALLED_APPS = [  
    # ... apps padrão do Django ...  
    'acervo', # nosso app  
]
```



Passo fácil de esquecer

Se o app não estiver em INSTALLED_APPS, o Django ignora seus models e templates.

Uma Primeira View

Só para ver o app respondendo

acervo/views.py

```
from django.http import HttpResponse

def inicio(request):
    return HttpResponse(
        'Olá, acervo!'
    )
```

acervo/urls.py (crie você mesmo)

```
from django.urls import path
from . import views

urlpatterns = [
    path('', views.inicio),
]
```

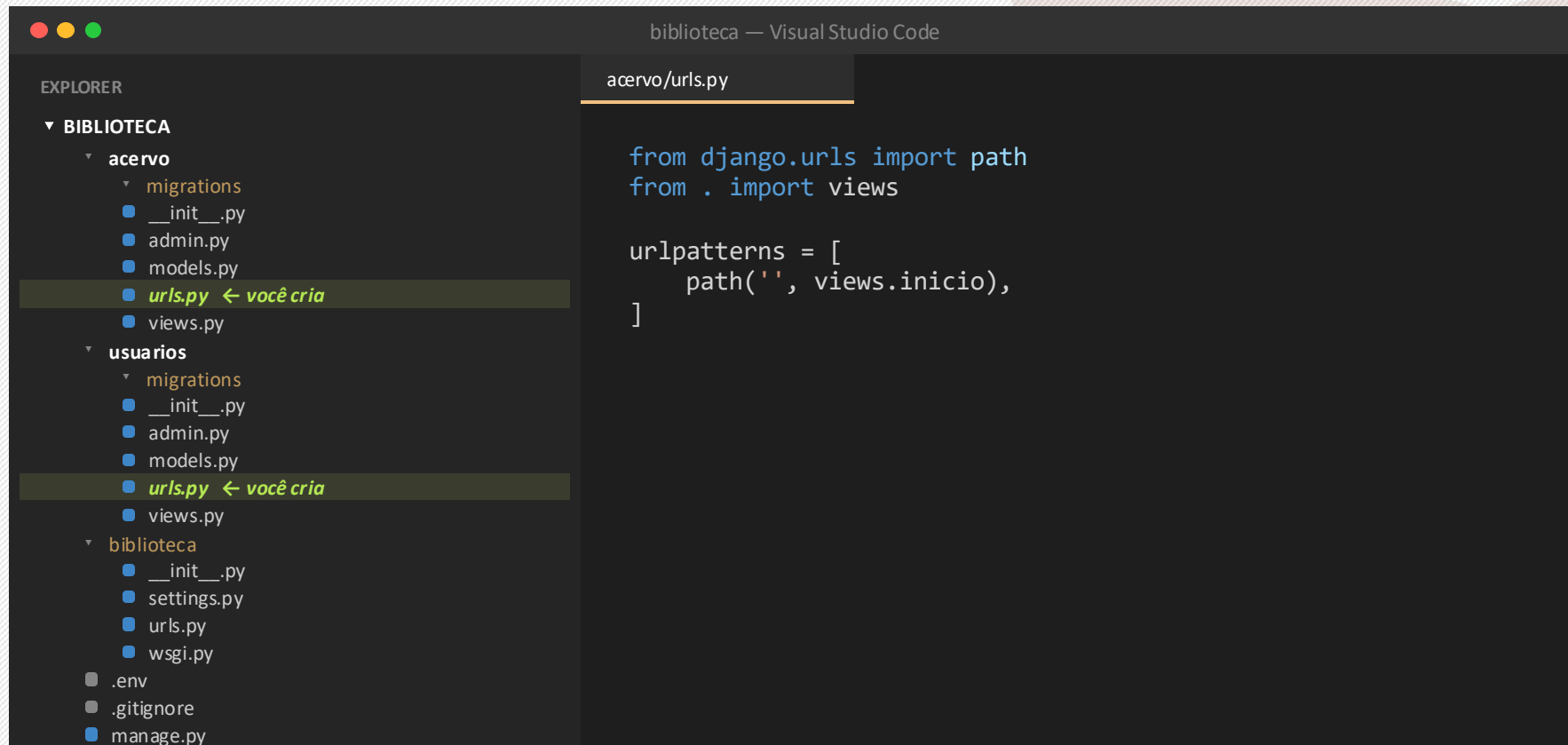


O que acabamos de ver — e um detalhe importante

A view é uma função que recebe a requisição e devolve uma resposta. O arquivo `acervo/urls.py` diz qual endereço chama essa função — mas ele **NÃO** vem pronto: o comando `startapp` não cria esse arquivo. Você precisa criá-lo manualmente dentro da pasta do app. Nas próximas aulas trocamos o texto por templates.

Um Projeto Tem Vários Apps

Simulação do VS Code: projeto "biblioteca" com 2 apps



Dois apps, mesma estrutura

acervo e usuarios são independentes: cada um tem seus próprios models, views e migrations — mas repetem o mesmo padrão de arquivos.

urls.py em destaque

Nos dois apps, o `urls.py` aparece destacado: é o arquivo que você cria manualmente (o startapp não o gera).

Passo 2

Modelar os Dados

Agora o coração da aula: os models

Um model descreve, em Python, como um dado é guardado. É o mapa das tabelas do seu banco.



Uma classe = uma tabela

Cada model é uma classe Python que vira uma tabela no banco. Cada atributo da classe vira uma coluna. Você descreve os dados uma vez e o Django cuida do resto.

Escrevendo o Model Livro

acervo/models.py

acervo/models.py

```
from django.db import models

class Livro(models.Model):
    titulo = models.CharField(max_length=200)
    autor = models.CharField(max_length=100)
    ano = models.IntegerField()
    disponivel = models.BooleanField(default=True)

    def __str__(self):
        return self.titulo
```



O método `__str__`

Define como o objeto aparece escrito — aqui, pelo título. Ajuda muito no Admin e no shell.

Os Tipos de Campo Mais Usados

Cada tipo de dado tem um Field

Field	Guarda	Exemplo
CharField	texto curto (com limite)	título, nome
TextField	texto longo	descrição, resumo
IntegerField	número inteiro	ano, quantidade
BooleanField	verdadeiro/falso	disponível?
DateField	data	data de publicação
DecimalField	número com casas	preço

Migrações: do Model ao Banco

Dois comandos que criam a tabela

terminal

```
# 1. gera o plano de mudança a partir dos models  
python manage.py makemigrations
```

```
# 2. aplica o plano no banco de dados  
python manage.py migrate
```



makemigrations

Lê seus models e escreve um arquivo descrevendo o que mudou — sem tocar no banco ainda.



migrate

Executa esse plano e cria/altera as tabelas de verdade no PostgreSQL.

O Ciclo das Migrações

Toda vez que mudar um model, repita

1

Edita o model

muda, adiciona ou remove um campo

2

makemigrations

o Django gera o plano da mudança

3

migrate

o plano é aplicado no banco

4

Pronto

banco e código de novo em sincronia



Regra de ouro

Mudou o model? Rode makemigrations e migrate. Esquecer disso é o erro nº 1 de quem começa — o código muda, mas o banco não acompanha.

Passo 3

O ORM na Prática

Conversando com o banco em Python puro

Com a tabela criada, o ORM deixa você criar, buscar e alterar registros sem escrever uma linha de SQL.



O shell do Django

O comando `python manage.py shell` abre um Python já conectado ao seu projeto. É o lugar ideal para testar o ORM e ver os dados aparecerem no banco na hora.

Criando e Buscando Registros

No shell do Django

```
python manage.py shell
```

```
>>> from acervo.models import Livro
```

```
# criar um livro
```

```
>>> Livro.objects.create(titulo='Dom Casmurro', autor='Machado', ano=1899)
```

```
# buscar todos
```

```
>>> Livro.objects.all()
```

```
<QuerySet [<Livro: Dom Casmurro>]>
```

```
# filtrar por condição
```

```
>>> Livro.objects.filter(ano=1899)
```



.objects e o gerente

Os Métodos do ORM que Você Mais Vai Usar

O CRUD em quatro verbos

Ação	Método do ORM	O que faz
Criar	<code>objects.create(...)</code>	insere um novo registro
Ler (todos)	<code>objects.all()</code>	retorna todos os registros
Ler (filtro)	<code>objects.filter(...)</code>	retorna os que batem na condição
Atualizar	<code>obj.save()</code>	salva as mudanças de um objeto
Apagar	<code>obj.delete()</code>	remove o registro do banco

Passo 4

O Django Admin

Um painel administrativo completo, de graça

Talvez o recurso mais amado do Django: uma interface pronta para cadastrar e gerenciar seus dados.



Por que isso é incrível

Sem escrever nenhuma tela, você ganha um sistema de login e um painel para criar, editar e apagar registros de qualquer model. Perfeito para alimentar o sistema enquanto as telas ainda não existem.

Ativando o Admin

Duas etapas rápidas

acervo/admin.py

```
from django.contrib import admin
from .models import Livro

admin.site.register(Livro)
```

terminal

```
# criar um usuário admin
python manage.py
    createsuperuser

# depois acesse:
/admin
```



O que acontece

Ao registrar o Livro no admin.py, ele aparece no painel. O createsuperuser cria seu login de administrador. Acesse <http://127.0.0.1:8000/admin> e cadastre livros pela interface — os dados vão direto para o PostgreSQL.

O Fluxo Completo de Hoje

Tudo o que você fez, em ordem

1

Ambiente

venv + Django

2

Projeto/App

startproject e startapp

3

Model

classe Livro

4

Migração

makemigrations + migrate

5

Admin

cadastrar dados



Você já tem um back-end funcional

Sem escrever HTML nem SQL, você tem dados guardados no banco e um painel para gerenciá-los. Na Aula 5 vamos construir as telas que o usuário final vê — com views e templates.

Comandos Essenciais do manage.py

Sua cola para o laboratório

Comando	O que faz
runserver	liga o servidor de desenvolvimento
startapp <nome>	cria um novo app
makemigrations	gera as migrações a partir dos models
migrate	aplica as migrações no banco
createsuperuser	cria um usuário administrador
shell	abre o Python conectado ao projeto

Conectando ao PostgreSQL

Trocando o banco padrão

O Django já vem com SQLite para testes. Para usar PostgreSQL, ajuste o settings.py:

settings.py

```
DATABASES = {  
    'default': {  
        'ENGINE': 'django.db.backends.postgresql',  
        'NAME': 'biblioteca_db', 'USER': 'postgres',  
        'PASSWORD': 'senha', 'HOST': 'localhost',  
    }  
}
```



Precisa do driver

Instale o conector com `pip install psycopg2-binary`. Depois, um migrate cria as tabelas já no PostgreSQL.

Credenciais no Código: um Problema Real

Por que `settings.py` não devia ter senhas

No slide anterior, a senha do banco ficou escrita direto no `settings.py`. Isso funciona — mas é uma prática perigosa que todo projeto profissional evita.



O que pode dar errado

Se você subir esse código para o GitHub (mesmo um repositório privado), a senha do banco vai junto, no histórico do Git, para sempre — mesmo que você a apague depois.



Credenciais são segredo

Senha de banco, chave secreta do Django (`SECRET_KEY`), tokens de API — nada disso deveria aparecer no código-fonte.



Cada ambiente, um valor

Localmente você usa um banco; em produção, outro. Valores fixos no código impedem essa flexibilidade.



A solução: variáveis de ambiente

Guardar esses valores FORA do código, em variáveis de ambiente — e carregá-las com a biblioteca `python-dotenv`. Vamos ver como.

python-dotenv: Escondendo as Credenciais

Três passos simples

terminal

```
pip install python-dotenv
```

.env (na raiz do projeto)

```
DB_NAME=biblioteca_db  
DB_USER=postgres  
DB_PASSWORD=senha  
DB_HOST=localhost  
SECRET_KEY=chave-secreta-django
```

settings.py

```
from dotenv import load_dotenv  
import os  
load_dotenv()  
DATABASES = {  
    'default': {  
        'NAME': os.getenv('DB_NAME'),  
        'PASSWORD': os.getenv('DB_PASSWORD'),  
    }  
}
```



Nunca versione o .env

Adicione .env ao arquivo .gitignore. Ele nunca deve ir para o GitHub — cada pessoa/ambiente tem o seu próprio.



Compartilhe um modelo

Crie um .env.example com os nomes das variáveis (sem valores reais) para orientar quem clonar o projeto.

Boas Práticas desde a Aula 4

Hábitos que evitam dor de cabeça



Sempre no venv

Ative o ambiente virtual antes de instalar ou rodar qualquer coisa.



Migrou a cada mudança

Editou model? makemigrations e migrate na sequência, sem acumular.



Sempre um __str__

Todo model merece um __str__ — o Admin e o shell ficam muito mais legíveis.



Commit a cada etapa

Ambiente pronto, model criado, admin funcionando — cada marco é um commit.

Erros Comuns Nesta Etapa

E como evitá-los



Esquecer INSTALLED_APPS

Criar o app e não registrá-lo no settings — o Django ignora tudo.



Não rodar migrate

Criar o model e esquecer de aplicar a migração: "no such table".



venv desativado

Instalar Django fora do venv e depois não achar o pacote.



Servidor parado

Editar o código e esquecer que o runserver precisa estar rodando para testar.

Glossário da Aula

Os termos que você leva desta aula

Termo	Significado em uma frase
venv	Ambiente virtual que isola as bibliotecas de um projeto Python
Projeto	O site inteiro — configurações e conjunto de apps
App	Módulo do projeto que cuida de uma funcionalidade
Migração	Plano que leva mudanças dos models para o banco de dados
Superuser	Usuário administrador com acesso total ao Django Admin

Resumo da Aula 4

O que você construiu hoje



Ambiente & projeto

venv, Django instalado, projeto e app criados e registrados.



Model & migração

O model Livro virou tabela no banco via makemigrations + migrate.



ORM & Admin

Registros criados pelo ORM e gerenciados pelo painel administrativo.



A conclusão da aula

Você saiu do zero e chegou a um back-end funcional: dados modelados em Python, guardados no PostgreSQL e gerenciáveis por um painel — tudo sem HTML nem SQL. Na Aula 5, damos telas a esse sistema.

Para a Próxima Aula

Aula 5 — Django na Prática II



Próxima aula: views, URLs e templates

Vamos transformar os dados do banco em páginas de verdade: escrever views que buscam os livros, mapear URLs e montar o HTML com templates. O usuário final finalmente vê o sistema.



Como se preparar

Deixe o projeto de hoje pronto e com alguns livros cadastrados no Admin — vamos exibi-los na tela na próxima aula.



Combinado

Já vá pensando no seu projeto: qual será o "Livro" dele? O cadastro central que tudo gira em torno?

Dúvidas?

Aula 4 — Django na Prática I

Obrigado!

Do terminal vazio a um sistema com dados — em uma aula.



Contato

marcio.garrido@univassouras.edu.br